

Design and Analysis of Algorithm

In theoretical analysis of algorithms, it is common to estimate their complexity in the asymptotic sense, i.e., to estimate the complexity function for arbitrarily large input. The term "**analysis of algorithms**" was coined by Donald Knuth.

Algorithm analysis is an important part of computational complexity theory, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem. Most algorithms are designed to work with inputs of arbitrary length. Analysis of algorithms is the determination of the amount of time and space resources required to execute it.

Usually, the efficiency or running time of an algorithm is stated as a function relating the input length to the number of steps, known as **time complexity**, or volume of memory, known as **space complexity**.

The Need for Analysis

In this chapter, we will discuss the need for analysis of algorithms and how to choose a better algorithm for a particular problem as one computational problem can be solved by different algorithms.

By considering an algorithm for a specific problem, we can begin to develop pattern recognition so that similar types of problems can be solved by the help of this algorithm.

Algorithms are often quite different from one another, though the objective of these algorithms are the same. For example, we know that a set of numbers can be sorted using different algorithms. Number of comparisons performed by one algorithm may vary with others for the same input. Hence, time complexity of those algorithms may differ. At the same time, we need to calculate the memory space required by each algorithm.

Analysis of algorithm is the process of analyzing the problem-solving capability of the algorithm in terms of the time and size required (the size of memory for storage while implementation). However, the main concern of analysis of algorithms is the required time or performance. Generally, we perform the following types of analysis –

- **Worst-case** – The maximum number of steps taken on any instance of size **a**.

- **Best-case** – The minimum number of steps taken on any instance of size **a**.
- **Average case** – An average number of steps taken on any instance of size **a**.
- **Amortized** – A sequence of operations applied to the input of size **a** averaged over time.

To solve a problem, we need to consider time as well as space complexity as the program may run on a system where memory is limited but adequate space is available or may be vice-versa. In this context, if we compare **bubble sort** and **merge sort**. Bubble sort does not require additional memory, but merge sort requires additional space. Though time complexity of bubble sort is higher compared to merge sort, we may need to apply bubble sort if the program needs to run in an environment, where memory is very limited.

Rate of Growth

Rate of growth is defined as the rate at which the running time of the algorithm is increased when the input size is increased.

The growth rate could be categorized into two types: linear and exponential. If the algorithm is increased in a linear way with an increasing in input size, it is **linear growth rate**. And if the running time of the algorithm is increased exponentially with the increase in input size, it is **exponential growth rate**.

Proving Correctness of an Algorithm

Once an algorithm is designed to solve a problem, it becomes very important that the algorithm always returns the desired output for every input given. So, there is a need to prove the correctness of an algorithm designed. This can be done using various methods –

Proof by Counterexample

Identify a case for which the algorithm might not be true and apply. If the counterexample works for the algorithm, then the correctness is proved. Otherwise, another algorithm that solves this counterexample must be designed.

Proof by Induction

Using mathematical induction, we can prove an algorithm is correct for all the inputs by proving it is correct for a base case input, say 1, and assume it is correct for another input k , and then prove it is true for $k+1$.

Proof by Loop Invariant

Find a loop invariant k , prove that the base case holds true for the loop invariant in the algorithm. Then apply mathematical induction to prove the rest of algorithm true.

